

Threaded Binary Trees

Single (One-Way) and Double (Two-Way) — Worked Examples

Unit III • Supplementary Examples

Part A

Single (right) threaded — 4 examples

Part B

Double (two-way) threaded — 4 examples

Single vs Double: what gets threaded

Single (right) threaded

Only RIGHT NULLs become threads.
Each points to the node's INORDER SUCCESSOR.
Left NULLs stay plain NULL.

Enables forward-only iterative inorder traversal.

Double (two-way) threaded

BOTH left and right NULLs become threads.
Right -> successor, Left -> PREDECESSOR too.

Enables traversal in EITHER direction from any node.

— real child - - - - thread

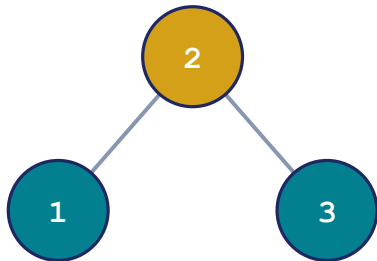
Every diagram ahead uses this convention: solid grey = real child pointer, dashed gold = thread. Watch for threads that RETRACE an existing solid edge — they still count, they are just not drawn as a second separate line.

Part A: Single (right) threading — the rule

- 1 Find every node whose RIGHT child is empty.
- 2 Point its right pointer at that node's INORDER SUCCESSOR instead of NULL.
- 3 The very LAST node in inorder has no successor — its right stays genuinely NULL.

Left pointers are UNTOUCHED — a NULL left stays NULL. That is the whole difference from double threading, coming in Part B.

Example 1: the simplest case — 3 nodes



inorder: 1 2 3

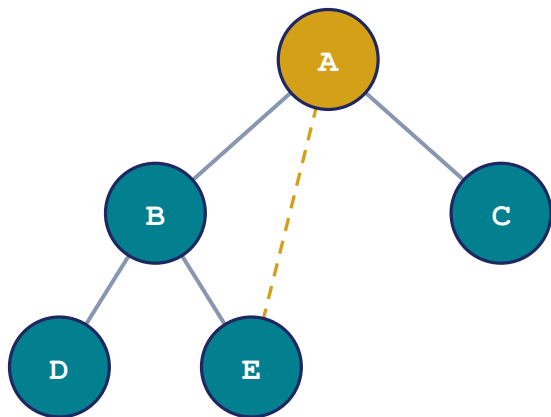
Right threads needed

Node 1 has no right child → successor is 2. Node 3 has no right child, but 3 is LAST in inorder → stays NULL.

Node 1's thread to 2 RETRACES the solid edge already drawn above (2 is 1's own parent) — no separate dashed line needed.

Total right threads in this tree: exactly ONE, and it is invisible in the diagram — already implied by the solid edge.

Example 2: a 5-node tree with one real thread line



inorder: D B E A C

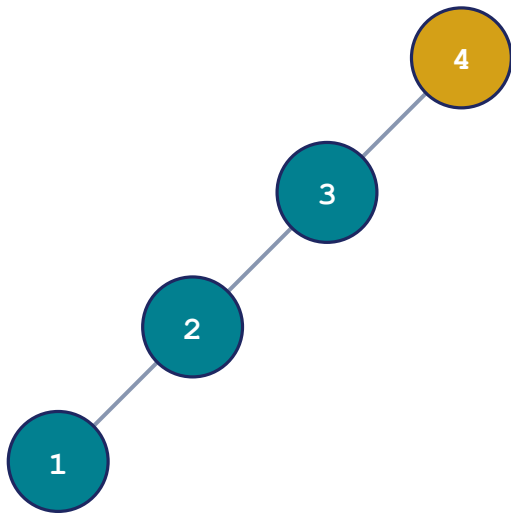
All three right threads

$D \rightarrow B$ (retraces the solid edge; D's own parent is B)

$E \rightarrow A$ (the ONE genuinely new dashed line — E's successor is not its parent B, it SKIPS past B up to A)

$C \rightarrow \text{NULL}$ (C is last in inorder)

Example 3: a skewed tree — every thread retraces the chain



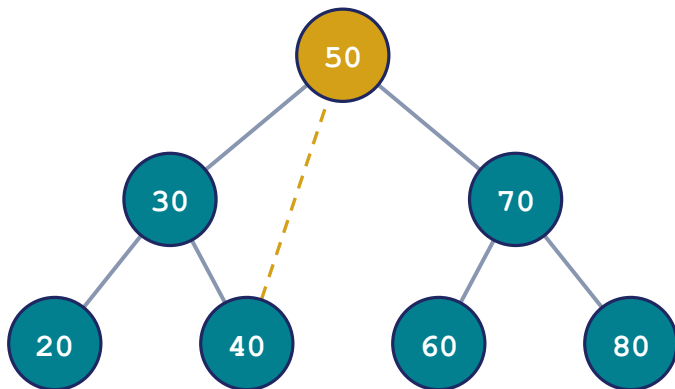
inorder: 1 2 3 4

Every single thread retraces a parent edge

- 1 $\rightarrow$ 2 (1's own parent)
- 2 $\rightarrow$ 3 (2's own parent)
- 3 $\rightarrow$ 4 (3's own parent)
- 4 $\rightarrow$ stays NULL (last in inorder)

Zero new dashed lines anywhere in this tree — every left-skewed chain threads "for free".

Example 4: a 7-node BST shape (values already sorted)



inorder: 20 30 40 50 60 70 80 (a BST's inorder is always sorted)

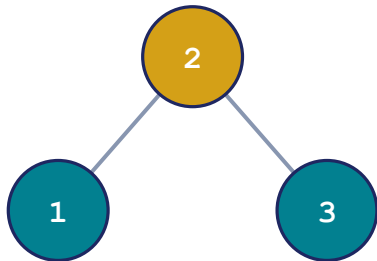
20→30 and 60→70 retrace parent edges. 40→50 is the ONE new dashed line — node 40's successor skips past its parent 30 to reach 50. 80 stays NULL (last in inorder).

Part B: Double threading — what gets ADDED

Everything from Part A stays exactly the same — every right thread is still there. Double threading ADDS one more rule: every empty LEFT pointer becomes a thread to the node's INORDER PREDECESSOR.

- The very FIRST node in inorder has no predecessor — its left stays genuinely NULL, mirroring how the last node's right stays NULL.
- A node with a real left child needs no left thread at all — exactly like Part A's rule for right children.
- Same four tree shapes, revisited — now watch what NEW dashed lines appear.

Example 1 revisited: the 3-node tree, now both ways



inorder: 1 2 3

All FOUR threads now (2 new)

Right threads (Part A, unchanged):

1 → 2 (retraces edge)

3 → stays NULL (last)

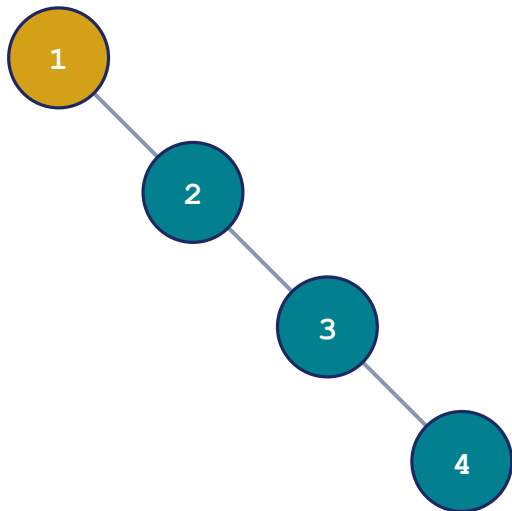
Left threads (NEW today):

1 → stays NULL (1 is FIRST in inorder)

3 → 2 (retraces edge — 3's own parent is 2)

Every thread in this tiny tree happens to retrace an edge or stay NULL — no separate dashed lines needed even now.

Example 2: right-skewed — the mirror image of Part A's chain



inorder: 1 2 3 4

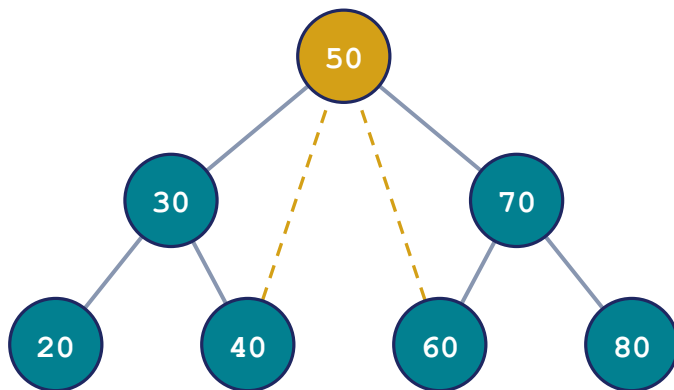
Left threads, mirroring Example 3's right threads

$2 \rightarrow 1$ $3 \rightarrow 2$ $4 \rightarrow 3$ (each retraces its own parent edge)

$1 \rightarrow$ stays NULL (1 is first in inorder)

Right threads: only node 4 needs one, and it stays NULL (last in inorder) — every other node already has a real right child.

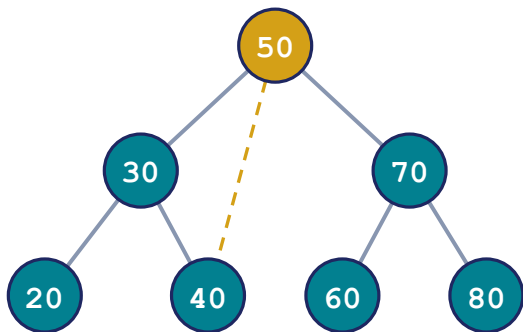
Example 3 revisited: the BST, now with a second skip thread



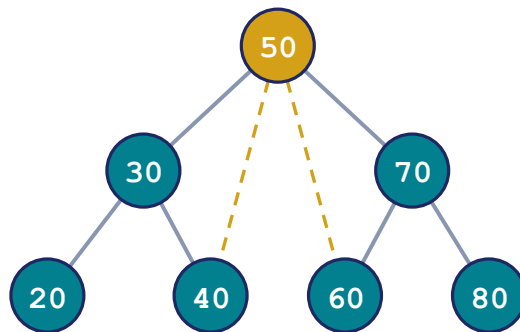
inorder: 20 30 40 50 60 70 80

The new dashed line: 60's LEFT thread skips past its parent 70 to reach 50 — the mirror partner of 40's right-thread skip from Example 4. Root 50 now has BOTH neighbours pointing directly at it.

Same tree, side by side — what doubling really adds



SINGLE (right) threaded — 1 visible thread



DOUBLE threaded — 2 visible threads

The right half of the tree gained NOTHING new in this view — $60 \rightarrow 70$ and $80 \rightarrow \text{NULL}$ already existed. The only change is the ONE new left thread, $60 \rightarrow 50$, appearing on the right diagram. Doubling a tree's threading does not double its thread COUNT; it only adds whatever left-side relationships were still missing.

Single vs Double — the trade-off

Aspect	Single (right) threaded	Double threaded
Threads added	Right NULLs only	Right AND left NULLs
Traversal directions	Forward (inorder) only	Forward AND backward
Finding a predecessor	Not directly possible	Direct — follow left thread/pointer
Extra flag bits per node	1 (rightThread)	2 (leftThread, rightThread)
Typical use	Simple forward iteration	Editors, undo/redo-style backward steps

SUMMARY

Five shapes, one consistent rule

- Single (right) threaded: only right NULLs become successor-threads. Left NULLs stay untouched.
- Double threaded: ADDS left-NULL-to-predecessor threads on top — not a different algorithm, an extra pass over the same rule.
- Many threads RETRACE an existing parent edge (skewed trees: every thread does). Only some SKIP past an intermediate node — those are the genuinely new dashed lines.
- The first node in inorder always keeps a real NULL on the left; the last node always keeps a real NULL on the right — in both single and double threading.

PRACTICE Draw a BST from values 15, 10, 20, 8, 12. Thread it (a) single, right only, then (b) double. Which threads retrace a parent edge, and which skip past one?